Part 1~4 보충: 직관적 이해를 위한 심화 해설

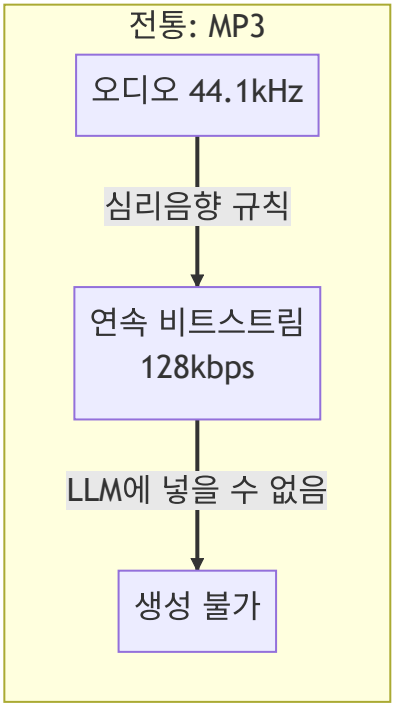
> Part 1(음악 AI), Part 2(로봇 AI), Part 3(시뮬레이션/RL/휴머노이드), Part 4(PyTorch 구현)에서 "왜 이 기법을 쓰는가?"가 비는 부분을 채웁니다.

Part 1: 음악 AI

1.1 Neural Audio Codec — 왜 MP3 대신 뉴럴넷으로 압축하는가?

전통 코덱(MP3)의 한계: MP3는 사람이 설계한 심리음향 규칙으로 압축한다. 압축 결과는 **연속적인 비트스트림**이다. 이것을 GPT 같은 언어 모델에 넣을 수 없다 — 언어 모델은 **이산 토큰**을 다루기 때문이다.

Neural Codec(EnCodec)의 핵심 동기: 오디오를 "이산 토큰의 시퀀스"로 변환하면, 음악 생성을 **텍스트 생성과 동일한 문제**로 환원할 수 있다.

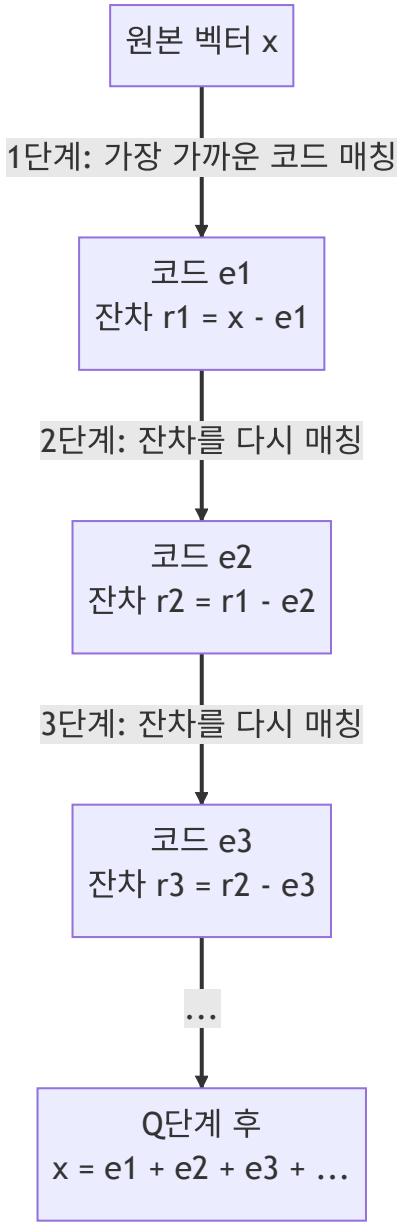


1.2 Residual Vector Quantization (RVQ) — 왜 코드북을 여러 겹 쌓는가?

단일 코드북의 한계: 코드북 크기 $K = 1024$이면 1024가지 패턴만 표현 가능. 음악의 디테일을 담기엔 부족하다.

코드북을 키우면? $K = 1024^8$이면 매칭할 때 10^{24}개를 비교해야 한다 → 불가능.

RVQ의 아이디어: "한 번에 정확하게 맞추지 말고, 여러 번에 걸쳐 점점 정교하게 맞추자."



- **1단계**: 대략적인 형태를 잡음 (가장 비슷한 "큰 분류")
- **2단계**: 1단계에서 못 맞춘 부분(잔차)을 보정
- **3~8단계**: 점점 미세한 디테일을 추가

비유: 초상화를 그릴 때 — 1단계에서 윤곽(얼굴형), 2단계에서 이목구비, 3단계에서 음영, 4단계에서 피부 결... 매 단계가 이전 단계의 **오차를 줄이는** 데 집중한다.

검색 비용: Q단계 × K개 코드 = $8 \times 1024 = 8192$번 비교 (vs 1024^8번)

1.3 Multi-Scale Discriminator — 왜 판별자가 여러 개 필요한가?

오디오에는 **여러 시간 스케일**의 구조가 있다:

스케일	구조	시간 범위
미세	파형의 개별 진동	~1ms
중간	음소, 음표	~50ms
거시	멜로디, 리듬 패턴	~1s

하나의 판별자가 모든 스케일을 동시에 평가하기 어렵다. 그래서:

- **Multi-Scale Discriminator (MSD)**: 원본, 2× 다운샘플, 4× 다운샘플 각각에 판별자
- **Multi-Period Discriminator (MPD)**: 주기 2, 3, 5, 7, 11으로 오디오를 2D로 재배열하여 주기적 패턴(피치, 하모닉스) 판별

1.4 STFT Loss — 왜 시간 영역 MSE로는 부족한가?

$$\text{시간 영역 MSE} = \|x(t) - \hat{x}(t)\|^2$$

문제: 시간 영역에서 같은 음이 1ms 밀려도 MSE가 크다. 하지만 사람 귀에는 거의 같게 들린다.

STFT Loss: 주파수 영역에서 비교하면 위상 이동에 덜 민감하다.

$$\mathcal{L}_{\text{STFT}} = \||\text{STFT}(x)| - |\text{STFT}(\hat{x})|\|_F$$

Multi-Resolution: FFT 크기를 달리하여 여러 해상도에서 비교:

- 작은 FFT (256): 시간 해상도 좋음 → 어택, 트랜지언트 포착
- 큰 FFT (2048): 주파수 해상도 좋음 → 음정, 하모닉스 포착

1.5 Feature Matching Loss — 왜 판별자의 중간 레이어를 비교하는가?

GAN 손실만 쓰면 학습이 불안정하다. 판별자의 최종 출력(진짜/가짜)은 이진적이라 그래디언트 정보가 빈약하다.

Feature Matching: 판별자의 **중간 레이어 활성화**를 비교한다.

$$\mathcal{L}_{\text{FM}} = \sum_{l=1}^{L} \frac{1}{N_l} \|D_l(x) - D_l(\hat{x})\|_1$$

중간 레이어는 다양한 추상 수준의 정보를 담고 있으므로, 더 풍부하고 안정적인 학습 신호를 제공한다.

비유: 미술 선생님이 "이건 가짜야" (최종 출력)라고만 말하면 도움이 안 된다. "윤곽선이 부드럽지 않고, 색감이 너무 균일하고, 질감이 플라스틱 같아" (중간 레이어)라고 말

해주면 개선 방향이 명확하다.

1.6 beta-VAE — 왜 KL 항의 가중치를 조절하는가?

VAE 손실: $\mathcal{L} = \text{Reconstruction} + \beta \cdot D_{KL}$

β	효과	용도
$\beta = 1$	원래 VAE (균형)	이론적 정석
$\beta < 1$	복원 강조, KL 약화	**오디오** (선명도 중요)
$\beta > 1$	KL 강조, 분리된 표현	**disentangled 표현** 학습

오디오 VAE에서 $\beta < 1$을 쓰는 이유: 오디오 복원에서 사소한 왜곡도 귀에 거슬린다. 잠재 공간의 "예쁜 구조"(KL=0)보다 **완벽한 복원**이 더 중요하다.

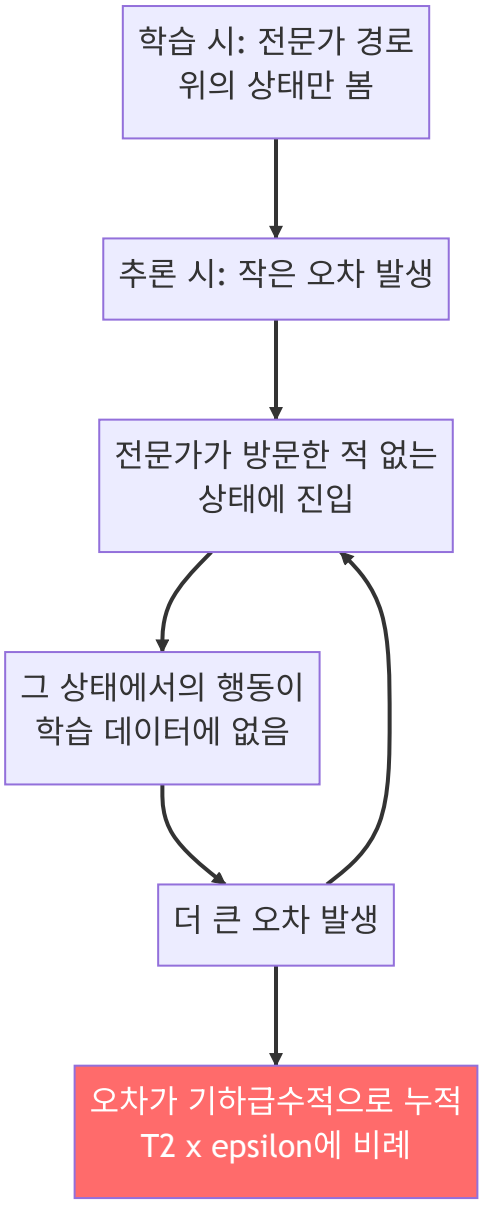
Part 2: 로봇 AI

2.1 Behavior Cloning — 왜 단순 모방만으로는 안 되는가?

Behavior Cloning (BC): 전문가의 (관찰, 행동) 쌍으로 지도학습

$$\min_{\theta} \mathbb{E}_{(s,a)\sim\mathcal{D}_{\text{expert}}}[\|a - \pi_\theta(s)\|^2]$$

치명적 문제: Distributional Shift



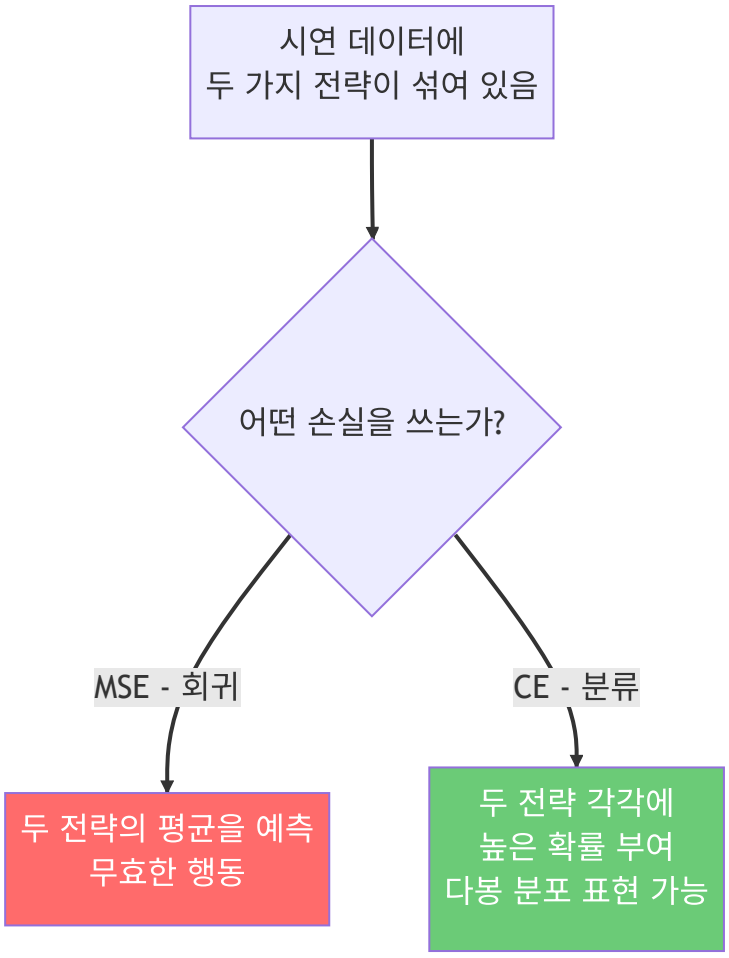
비유: 자동차 운전을 교관 옆에서만 배운 학생이, 혼자 운전하다 약간 차선을 벗어남 → 당황 → 더 벗어남 → 교관이 그 상황에서 어떻게 하는지 배운 적 없음 → 사고

이것이 **순수 모방학습의 근본 한계**이며, RL 파인튜닝이 필요한 이유다.

2.2 Action Discretization — 왜 로봇 행동을 256칸으로 나누는가?

연속 행동 $a \in [-1, 1]$을 256개 구간으로 양자화하여 **분류 문제**로 바꾼다.

왜 회귀(MSE) 대신 분류(CE)를 쓰는가?



예시: 컵을 잡을 때 "왼쪽에서" 또는 "오른쪽에서" 잡는 두 가지 시연이 있다면:

- **MSE**: 평균인 "정면에서" 잡는 행동 예측 → 컵을 놓칠 수 있음
- **CE (256-bin)**: "왼쪽"과 "오른쪽" 두 빈에 높은 확률 → 둘 중 하나를 샘플링

이것은 PixelRNN이 256-way softmax를 쓰는 이유와 정확히 같다.

2.3 FiLM — 왜 언어 명령을 "곱셈과 덧셈"으로 주입하는가?

$$\mathrm{FiLM}(x, \gamma, \beta) = \gamma \cdot x + \beta$$

여기서 γ, β는 텍스트 명령에서 생성된다.

대안들과 비교:

방법	동작	한계
입력에 텍스트 연결 (concatenation)	첫 층에서만 조건 주입	깊은 층까지 영향 약함
덧셈 (addition)	$x + \mathrm{text_emb}$	이동(shift)만 가능

FiLM	$\gamma \cdot x + \beta$	**스케일 + 이동** 둘 다 가능

왜 곱셈이 중요한가? γ가 0에 가까우면 그 채널을 **꺼버릴** 수 있고, 2에 가까우면 **강조**할 수 있다. 이것은 "빨간 컵을 집어"라는 명령에서 빨간색 채널을 강조하고, 파란색 채널을 억제하는 것과 같다.

이것은 StyleGAN의 AdaIN과 **정확히 같은 원리**다. AdaIN도 피처맵에 스케일(y_s)과 이동(y_b)을 적용한다.

2.4 Token Learner — 왜 81개 토큰을 8개로 줄이는가?

이미지를 ViT에 넣으면 $9 \times 9 = 81$개 패치 토큰이 나온다. Transformer의 Self-Attention은 $O(N^2)$이므로 토큰이 많으면 비용이 폭발한다.

Token Learner: 81개 토큰 중 **중요한 8개만 선택**

각 토큰은 학습된 공간적 어텐션 맵으로 선별된다:

- 토큰 1: 그리퍼(로봇 손)에 집중
- 토큰 2: 목표 물체에 집중
- 토큰 3: 장애물에 집중
- ...

$81 \to 8$이면 Attention 비용이 $81^2/8^2 \approx 100$배 감소. 실시간 로봇 제어에 필수적.

2.5 Flow Matching for Actions — 왜 Diffusion 대신 Flow Matching을 쓰는가?

pi0 모델이 로봇 행동 생성에 Flow Matching을 사용하는 이유:

	Diffusion	**Flow Matching**
수학적 기반	SDE (확률 미분방정식)	ODE (상미분방정식)
생성 경로	구불구불	**직선**
필요 스텝	50~1000	**8~10**
로봇 제어 적합성	느림	**실시간 가능**

로봇에서 속도가 왜 중요한가? 로봇 제어 루프는 보통 10~50Hz로 돌아간다. 0.1초 안에 행동을 결정해야 한다. Diffusion의 50스텝은 너무 느리지만, Flow Matching의 10스텝은 충분히 빠르다.

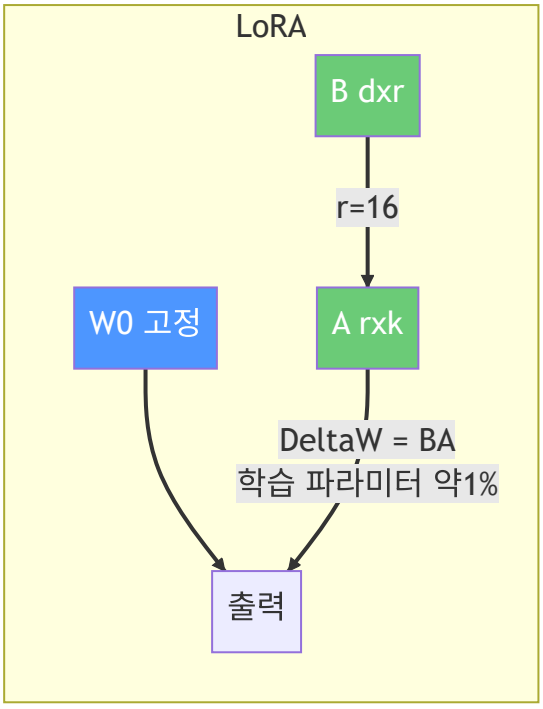
2.6 LoRA — 왜 7B 파라미터를 전부 학습하지 않는가?

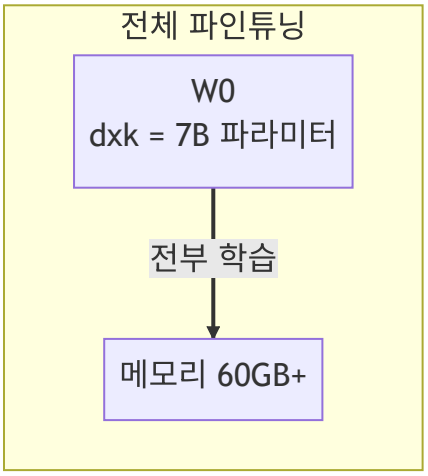
문제: 7B 모델 전체를 파인튜닝하면 GPU 메모리 ~60GB 필요 + 사전학습 지식 손실 위험

LoRA의 핵심 가정: "파인튜닝 시 가중치 변화 ΔW는 **저랭크(low-rank)**이다"

$$W' = W_0 + \Delta W = W_0 + BA$$

여기서 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$





왜 저랭크가 합리적인가? 사전학습된 모델은 이미 범용적 지식을 갖고 있다. 로봇 데이터로 파인튜닝할 때 바뀌는 것은 "로봇 특화 행동 패턴" 정도이며, 이것은 전체 가중치 공간의 **저차원 부분공간**에 해당한다.

B를 0으로 초기화하는 이유: $\Delta W = BA = 0$이므로, 학습 시작 시점에 원래 모델과 동일하다. 점진적으로 로봇 데이터에 맞게 변화한다.

2.7 VLM → VLA — 왜 언어 모델로 로봇을 제어하는가?

핵심 통찰 (RT-2): 로봇 행동도 "토큰"이다.

```
일반 VLM:  [이미지 토큰] [텍스트 토큰] → [다음 텍스트 토큰]
VLA:       [이미지 토큰] [명령 토큰]  → [행동 토큰]
```

행동 토큰 = 이산화된 행동값 (예: $\Delta x = 0.05$ → 토큰 #137)

왜 이것이 강력한가? 웹 스케일 데이터로 사전학습된 VLM의 **시각적 이해력과 언어적 추론 능력**을 로봇 제어에 그대로 활용할 수 있다. "빨간 컵을 집어"라는 명령에서 "빨간 컵"이 무엇인지, "집는다"가 무엇인지를 VLM이 이미 알고 있다.

Part 3: 시뮬레이션, RL, 휴머노이드

3.1 Sim-to-Real — 왜 시뮬레이션에서 학습하는가?

실제 로봇으로 학습하면?

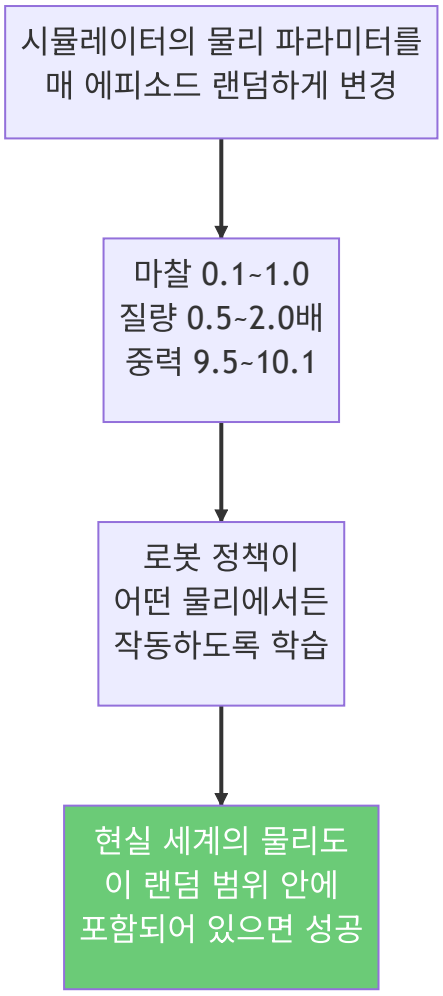
- 로봇이 부서질 수 있다
- 데이터 수집이 느리다 (실시간)
- 위험한 행동을 시도할 수 없다

시뮬레이션의 장점: 빠르고, 안전하고, 병렬화 가능

문제: Domain Gap — 시뮬레이터는 현실이 아니다

$$\text{시뮬레이션에서 잘 되는 정책} \neq \text{현실에서 잘 되는 정책}$$

3.2 Domain Randomization — 왜 일부러 환경을 "망가뜨리는가"?



비유: 다양한 비포장도로에서 운전 연습을 하면, 처음 보는 도로에서도 운전할 수 있다. 매끄러운 도로에서만 연습하면, 약간의 요철에도 당황한다.

핵심 가정: 현실 물리의 파라미터가 랜덤화 범위 안에 있어야 한다. 만약 현실의 마찰이 0.01인데 랜덤화 범위가 0.1~1.0이면 실패한다.

3.3 PPO — 왜 이 특정 RL 알고리즘을 쓰는가?

정책 그래디언트의 문제: 한 번의 업데이트로 정책이 너무 크게 바뀔 수 있다. 로봇이 갑자기 위험한 행동을 하기 시작할 수 있다.

PPO의 해결: 정책 변화량을 **클리핑**한다.

$$L^{\text{CLIP}} = \min\left(r(\theta) \cdot A,\ \text{clip}(r(\theta), 1-\epsilon, 1+\epsilon) \cdot A\right)$$

여기서 $r(\theta) = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}$는 확률 비율.

직관:

- $r = 1$: 정책 안 바뀜

- $r = 1.5$: 이 행동의 확률이 50% 증가
- 클리핑: r을 $[1-\epsilon, 1+\epsilon]$ 범위로 제한 → 한 번에 너무 많이 안 바뀜

왜 TRPO 대신 PPO? TRPO는 2차 최적화(Hessian)가 필요해 비용이 크다. PPO는 간단한 클리핑으로 비슷한 효과를 달성한다. 구현이 훨씬 쉽고 빠르다.

3.4 GAE — 왜 편향-분산 트레이드오프가 중요한가?

RL에서 "이 행동이 얼마나 좋았는가?" (A, Advantage)를 추정해야 한다.

방법	편향	분산	문제
Monte Carlo ($\lambda = 1$)	없음	**높음**	학습이 느리고 불안정
TD(0) ($\lambda = 0$)	**높음**	낮음	가치 함수가 부정확하면 오류 전파
GAE ($\lambda = 0.95$)	약간	약간	**최적 절충**

$$A^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}$$

λ가 "미래를 얼마나 멀리 볼 것인가"를 결정한다. $\lambda = 0.95$는 "미래를 많이 보되, 아주 먼 미래는 할인한다"는 뜻.

3.5 Reward Shaping — 왜 보상 설계가 중요한가?

Sparse Reward: "컵을 잡았으면 +1, 아니면 0"

문제: 로봇이 무작위로 움직여서 우연히 컵을 잡을 확률 ≈ 0. 학습 신호가 전혀 없다.

Dense Reward: "컵에 가까워질수록 +, 컵을 잡으면 큰 +"

Potential-Based Shaping의 핵심 정리 (Ng et al., 1999): $\gamma\Phi(s') - \Phi(s)$ 형태로 보상을 추가하면, **최적 정책이 변하지 않는다.**

이것이 중요한 이유: 아무렇게나 보상을 추가하면 "보상 해킹" (진짜 목표 대신 보상의 허점을 찾는 행동)이 발생한다. Potential-based 형태만이 안전하다.

3.6 ZMP와 캡처 포인트 — 왜 이족 보행이 어려운가?

근본 문제: 두 발로 서는 것 자체가 **불안정한 균형**이다. 네 발 로봇은 정적으로 안정적(지지 다각형이 넓음)이지만, 이족 로봇은 지지 다각형이 좁아서 항상 넘어질 위험이 있다.

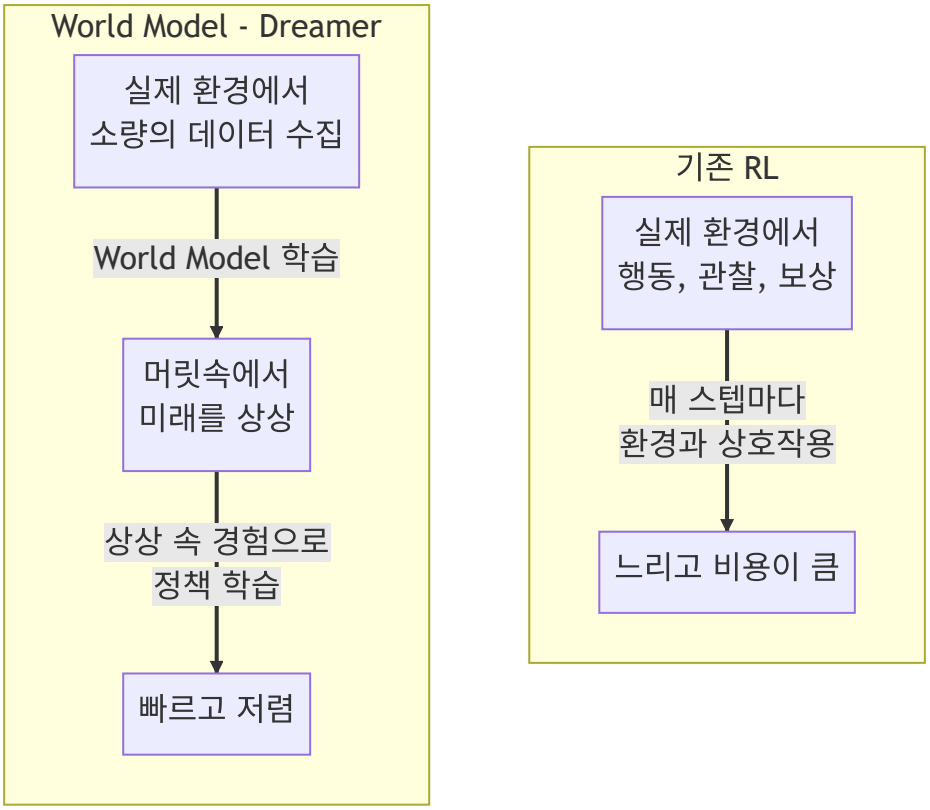
ZMP (Zero Moment Point): 지면 반력의 모멘트가 0인 점. 이 점이 **발바닥 영역(지지 다각형) 안에** 있으면 안 넘어진다.

캡처 포인트: "지금 한 발을 여기에 딛으면 넘어지지 않는 위치"

$$p_{\text{foot}} = p_{\text{CoM}} + \frac{v_{\text{CoM}}}{\omega}$$

무게 중심의 위치 + 속도에 비례하는 보정. 빨리 움직이고 있으면 더 앞에 발을 딛어야 한다. 이것은 사람이 달리다 급정거할 때 자연스럽게 하는 것과 같다.

3.7 World Model (Dreamer) — 왜 "상상"으로 학습하는가?



RSSM (Recurrent State-Space Model)의 두 가지 경로:

- **결정적 경로** (GRU): 장기 기억 유지
- **확률적 경로** (잠재 z): 불확실성 표현

왜 두 경로 모두 필요한가? 결정적 경로만 있으면 "세상은 확정적이다"라고 가정하는 셈 — 동전 던지기를 예측할 수 없다. 확률적 경로만 있으면 장기 기억이 약하다. 둘을 합치면 "기억하면서도 불확실한 것은 불확실하다고 인정"하는 모델이 된다.

Part 4: PyTorch 구현

4.1 log(var) 파라미터화 — 왜 σ 대신 $\log \sigma^2$를 출력하는가?

문제: 분산 σ^2는 양수여야 한다. 뉴럴넷의 출력은 임의의 실수.

해결 방법들:

방법	양수 보장	문제
$\sigma^2 = \lvert x \rvert$	✓	$x = 0$에서 미분 불가
$\sigma^2 = \mathrm{softplus}(x)$	✓	수치적으로 불안정할 수 있음
$\sigma^2 = \exp(\mathrm{logvar})$	✓	**가장 안정적**

logvar를 쓰는 이유:

1. $\exp$는 항상 양수 → 양수 보장
2. KL divergence 공식에 $\log \sigma^2$가 직접 등장 → 변환 불필요
3. σ^2가 매우 작을 때도 $\log \sigma^2$는 유한한 값 → 수치 안정

4.2 Sinusoidal Time Embedding — 왜 정수 t를 사인/코사인으로 변환하는가?

Diffusion U-Net은 타임스텝 t에 따라 다른 수준의 노이즈 제거를 해야 한다. 정수 t를 그냥 넣으면?

문제: $t = 500$과 $t = 501$은 거의 같은 수준의 노이즈인데, 숫자 500과 501은 뉴럴넷에 큰 의미가 없다. 또한 $t = 1$과 $t = 1000$ 사이의 관계가 선형적이지 않다.

Sinusoidal Embedding:

$$\mathrm{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \mathrm{PE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right)$$

- 다양한 주파수의 사인/코사인 → t의 **다중 스케일** 표현
- 비슷한 t → 비슷한 임베딩 (연속성)
- 먼 t → 다른 임베딩 (구별력)
- Transformer의 위치 인코딩과 동일한 원리

4.3 register_buffer — 왜 일반 텐서가 아니라 buffer로 등록하는가?

```
self.register_buffer('betas', torch.linspace(1e-4, 0.02, T))
```

일반 텐서와의 차이:

	`self.x = tensor`	`self.register_buffer('x', tensor)`
`model.to('cuda')` 시 이동	❌ (수동 이동 필요)	✅ (자동 이동)
`model.state_dict()` 포함	❌	✅ (저장/로드 가능)

옵티마이저 업데이트	❌	❌ (학습 안 됨)

노이즈 스케줄 상수($\beta_t, \bar{\alpha}_t$ 등)는 **학습은 안 되지만 모델과 함께 이동하고 저장되어야** 하므로 buffer가 딱 맞다.

4.4 Gradient Clipping — 왜 기울기를 잘라내는가?

$$\text{if } \|g\| > \text{max_norm}: \quad g \leftarrow g \cdot \frac{\text{max_norm}}{\|g\|}$$

문제: 가끔 미니배치에서 비정상적으로 큰 기울기가 발생한다 (이상치 데이터, 수치적 불안정). 이것이 파라미터를 크게 변화시켜 학습을 망친다.

해결: 기울기 벡터의 L2 노름이 임계값(보통 1.0)을 넘으면, **방향은 유지하되 크기를 줄인다**.

비유: 네비게이션이 "3km 직진 후 좌회전"이라고 해도 시속 300km로 달리진 않는다. 방향은 맞되, 속도를 제한하는 것.

4.5 Mixed Precision (AMP) — 왜 FP16과 FP32를 섞어 쓰는가?

	FP32	FP16
메모리	32비트	16비트 (**절반**)
속도	기준	**~2배 빠름**
정밀도	높음	낮음 (작은 값 소실 가능)

AMP의 전략:

- 순전파/역전파: FP16 (빠르게)
- 파라미터 업데이트: FP32 (정밀하게)
- GradScaler: Loss를 크게 키워서(scale up) FP16의 소수점 이하 소실을 방지, 업데이트 시 다시 줄임(scale down)

왜 이것이 작동하는가? 대부분의 연산은 높은 정밀도가 불필요하다. 하지만 파라미터 업데이트(작은 learning rate × 작은 기울기)는 정밀도가 필요하다. AMP는 "필요한 곳에만" 정밀도를 투자한다.

4.6 EMA (Exponential Moving Average) — 왜 학습된 가중치의 "평균"을 쓰는가?

$$\theta_{\text{EMA}} \leftarrow \alpha \cdot \theta_{\text{EMA}} + (1 - \alpha) \cdot \theta$$

보통 $\alpha = 0.9999$ (0에 매우 가까운 학습률)

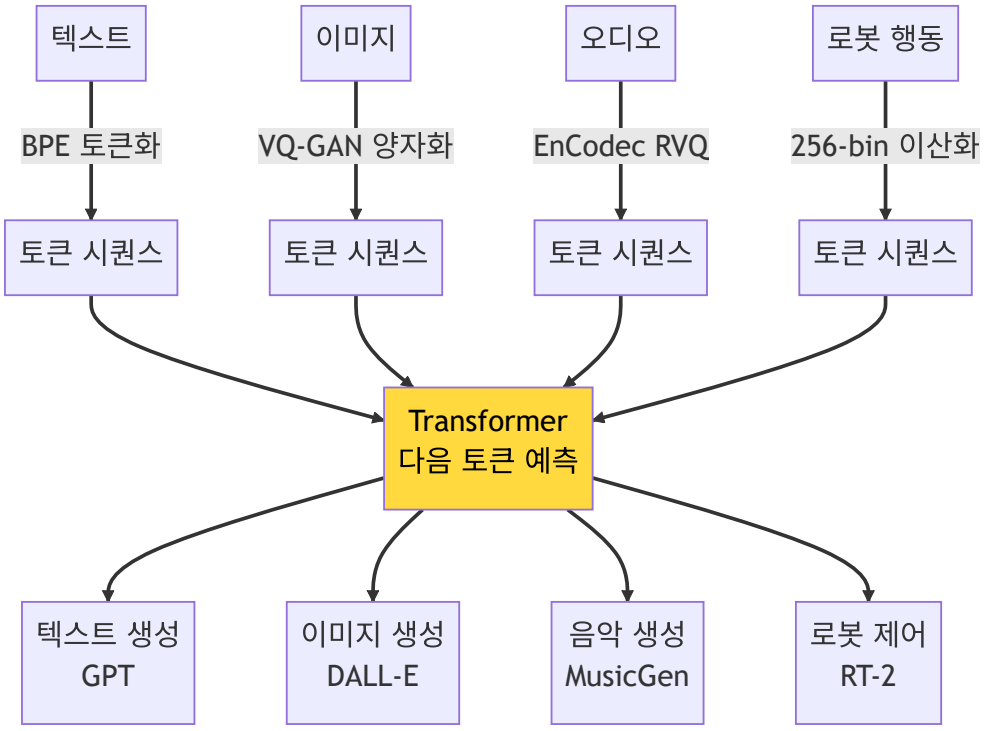
문제: 학습 중 가중치가 진동한다 (미니배치마다 다른 방향의 기울기). 특정 시점의 가중치는 운이 좋거나 나쁠 수 있다.

EMA의 효과: 최근 수천 스텝의 가중치를 부드럽게 평균하여, 진동을 줄인 안정적인 가중치를 얻는다.

비유: 주가의 일일 변동보다 20일 이동평균이 추세를 더 잘 보여주는 것과 같다. Diffusion 모델에서 EMA는 생성 품질을 상당히 향상시킨다.

전체 Part를 관통하는 핵심 패턴

패턴: "토큰화 → Transformer → 생성"의 범용성

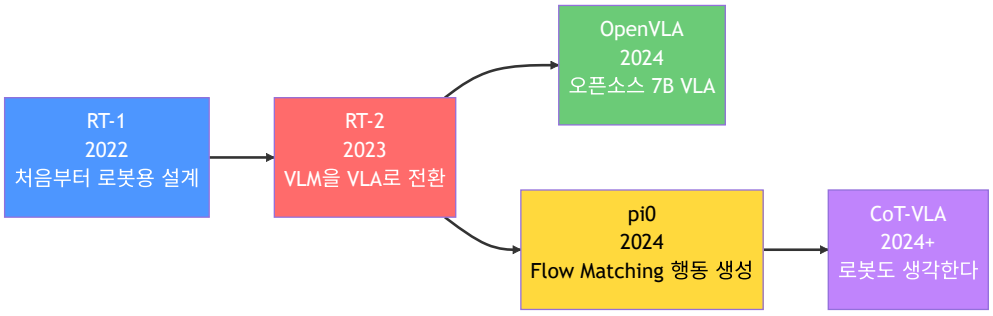


핵심 메시지: 어떤 데이터든 "이산 토큰의 시퀀스"로 변환하면, **언어 모델의 아키텍처와 학습 기법**을 그대로 적용할 수 있다. 이것이 현재 AI 연구의 가장 강력한 통합 원리이다.

보충: 빠진 핵심 주제들

Part 2 보충: VLA 모델 아키텍처 계보

로봇 AI의 핵심은 **"어떻게 시각 정보와 언어 명령을 로봇 행동으로 변환하는가"**이다. 이 절에서는 Vision-Language-Action(VLA) 모델의 진화 과정을 추적한다. RT-1에서 시작하여 RT-2, OpenVLA, pi0, CoT-VLA로 이어지는 계보를 통해, "행동은 또 다른 언어"라는 핵심 통찰이 어떻게 발전했는지를 이해한다.



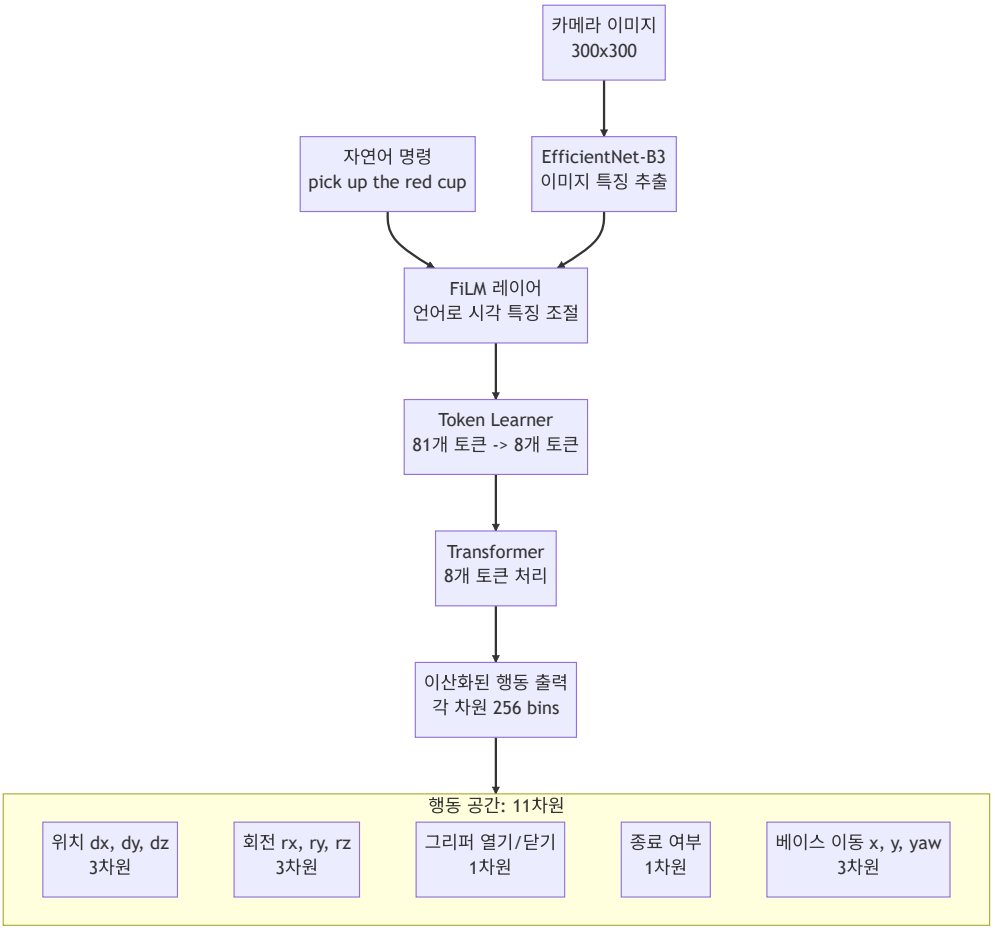
2.8 RT-1 — 최초의 대규모 로봇 Transformer

RT-1(Robotics Transformer 1)은 Google DeepMind가 2022년에 발표한, **대규모 데이터 + 대형 모델** 조합을 로봇 제어에 처음으로 성공적으로 적용한 모델이다.

핵심 통찰: 스케일이 중요하다. 더 많은 데이터 + 더 큰 모델 = 더 나은 일반화. 이것은 NLP에서 GPT 시리즈가 보여준 스케일링 법칙이 로봇에도 적용된다는 것을 증명했다.

데이터 규모: 13대의 실제 로봇으로 17개월에 걸쳐 수집한 **130,000개의 시연** 데이터. 700개 이상의 서로 다른 과제를 포함한다.

아키텍처 파이프라인:



행동 공간의 구조:

- **7-DoF 팔 제어**: 위치 3차원($\Delta x, \Delta y, \Delta z$) + 회전 3차원($\Delta r_x, \Delta r_y, \Delta r_z$) + 그리퍼 1차원
- **베이스 이동**: x, y 방향 이동 + θ 회전
- **종료 토큰**: 과제 완료 시 에피소드 중단
- **각 차원을 256개 구간(bin)으로 이산화** → 분류 문제로 변환

FiLM-Conditioned EfficientNet: 이미지 인코더인 EfficientNet의 각 블록에 FiLM 레이어를 삽입한다. 이것은 2.3절에서 설명한 것과 같은 원리로, 언어 명령이 시각 특징의 **어떤 채널을 강조/억제할지**를 결정한다.

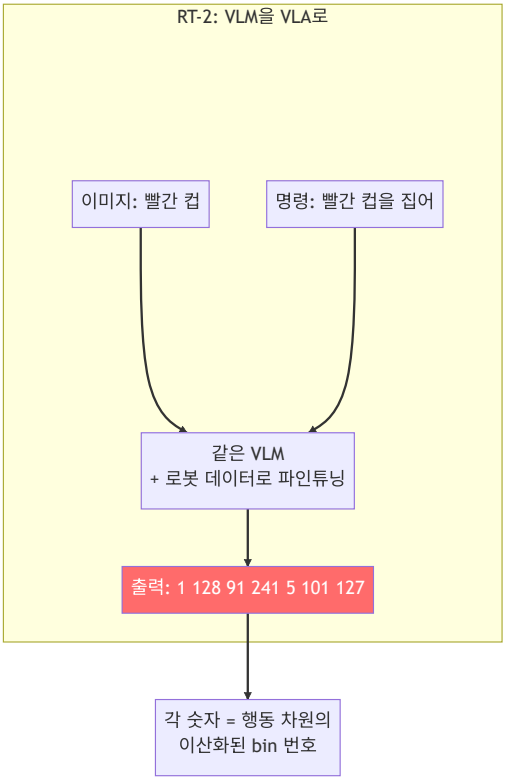
Token Learner의 역할: EfficientNet의 출력은 $9 \times 9 = 81$개의 공간 토큰이다. Token Learner가 이것을 **8개의 요약 토큰**으로 압축한다. 2.4절에서 설명한 것처럼, 이것은 Transformer의 $O(N^2)$ 비용을 약 100배 줄여서 **실시간 제어(3Hz)**를 가능하게 한다.

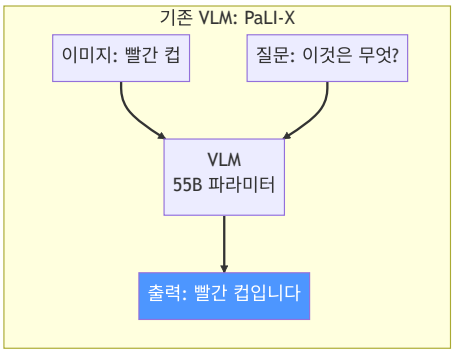
RT-1의 한계: 아무리 데이터가 많아도 "닫힌 세계(closed world)"에서만 작동한다. 학습 시 본 적 없는 물체나 환경에는 일반화가 어렵다. 이 한계를 극복하기 위해 **웹 스케일 지식**을 활용하는 RT-2가 등장한다.

2.9 RT-2 — VLM을 VLA로 전환하는 핵심 아이디어

RT-2는 VLA 모델 계보에서 가장 중요한 **패러다임 전환**을 이룬 모델이다. 핵심 돌파구는 놀랍도록 단순하다: **"행동은 또 다른 언어일 뿐이다."**

아이디어의 본질: PaLI-X(55B 파라미터의 거대 VLM)는 이미 이미지를 보고 텍스트를 생성할 수 있다. "이 이미지에 무엇이 있나요?" → "빨간 컵이 있습니다." 그렇다면 출력을 약간만 바꿔서 — 텍스트 대신 **행동 토큰**을 생성하면 어떨까?





행동의 텍스트 인코딩: 로봇의 7차원 행동 $[a_1, a_2, ..., a_7]$을 각각 256개 구간으로 이산화한 후, **텍스트 토큰으로 변환**한다.

예: 행동 $[\Delta x = 0.01, \Delta y = -0.05, \Delta z = 0.1, ...]$ → 이산화 → $[1, 128, 91, 241, 5, 101, 127]$

이 숫자들은 모델의 기존 어휘(vocabulary)에 이미 있는 토큰을 재활용한다. 모델 입장에서는 "빨간 컵입니다"를 출력하는 것과 "1 128 91 241 5 101 127"을 출력하는 것이 **구조적으로 동일한 연산**이다.

```python
# RT-2 의사코드: VLM을 VLA로 변환
class RT2(VLM):
    def tokenize_action(self, action):
        # 연속 행동 [-1, 1]을 256 구간으로 이산화
        bins = ((action + 1) / 2 * 255).int()  # [-1,1] → [0,255]
        # 각 차원을 텍스트 토큰으로 변환 (기존 어휘의 일부 사용)
        tokens = [self.vocab[256 + b] for b in bins]
        return tokens

    def forward(self, image, instruction):
        # 이미지 + 지시문을 토큰으로 인코딩 (기존 VLM과 동일)
        vision_tokens = self.vision_encoder(image)
        text_tokens = self.tokenizer(instruction)
        # Transformer가 다음 토큰 예측 → 행동 토큰 출력
        output = self.transformer([vision_tokens, text_tokens])
        action = self.detokenize_action(output[-7:])  # 마지막 7개 =
        return action
```

왜 이것이 작동하는가? VLM은 수십억 장의 웹 이미지와 텍스트로 사전학습되면서 이미 다음을 알고 있다:

- "빨간 컵"이 어떻게 생겼는지 (시각적 접지)
- "집는다"가 무슨 의미인지 (언어적 이해)
- 물체의 위치와 방향 (공간적 추론)

파인튜닝에서 추가로 배우는 것은 **"이 개념들을 어떤 모터 명령으로 변환하는가"** 뿐이다.

Co-Fine-Tuning: 망각 방지 전략: 로봇 데이터만으로 파인튜닝하면 사전학습 지식을 잊어버린다(catastrophic forgetting). RT-2는 **웹 데이터와 로봇 데이터를 50:50으로 섞어서** 동시에 학습한다. 이것은 ChatGPT가 SFT(Supervised Fine-Tuning) 시 기존 능력을 유지하기 위해 사전학습 데이터를 섞는 것과 같은 원리다.

창발적 능력 — 학습하지 않은 것을 할 수 있다: RT-2의 가장 놀라운 결과는 로봇 데이터에 **한 번도 등장하지 않은 명령**을 수행할 수 있다는 것이다.

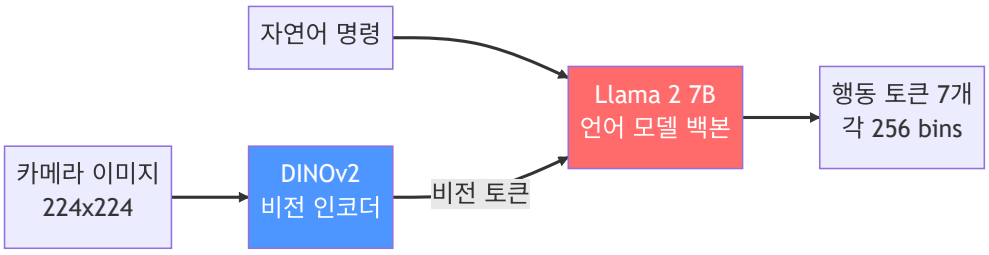
명령	로봇 데이터에 있었나?	성공 여부
"빨간 컵을 집어"	O	O
"바나나를 재배 국가에 놓아"	X	O (지도 위에 놓음)
"테일러 스위프트의 국적 깃발을 집어"	X	O (미국 깃발 선택)
"코카콜라보다 건강한 음료를 집어"	X	O (물병 선택)

이것이 가능한 이유: VLM이 사전학습에서 "바나나 = 열대과일 = 에콰도르/필리핀" 같은 **세계 지식**을 이미 갖고 있기 때문이다. 로봇 데이터는 "물체를 집어서 어디에 놓는" 운동 기술만 가르치면 된다.

2.10 OpenVLA — 오픈소스 7B VLA

RT-2는 55B 파라미터의 거대 모델로, 접근성이 낮다. OpenVLA는 이 아이디어를 **7B 규모로 축소**하고 **오픈소스**로 공개한 모델이다.

아키텍처 구성:



RT-2와의 핵심 차이점:

	RT-2	OpenVLA
모델 크기	55B (PaLI-X)	**7B** (Llama 2)
GPU 요구	TPU 다수	**단일 GPU** 가능
비전 인코더	PaLI ViT	**DINOv2** (자기지도학습)
학습 데이터	Google 내부	**Open X-Embodiment** (공개)
코드/가중치	비공개	**완전 오픈소스**

Open X-Embodiment 데이터셋: 22종의 서로 다른 로봇에서 수집한 **970,000개 에피소드**. 다양한 로봇 형태(팔, 이동 로봇 등)와 과제를 포함한다. 이것은 "로봇계의 ImageNet"을 목표로 한 데이터셋이다.

DINOv2를 쓰는 이유: DINOv2는 자기지도학습(self-supervised learning)으로 사전학습된 비전 인코더로, 레이블 없이도 **물체의 공간적 구조**를 잘 이해한다. PaLI의 ViT보다 더 정밀한 공간 이해가 가능하여 로봇 매니퓰레이션에 유리하다.

LoRA로 새 로봇 적응: OpenVLA의 가장 실용적인 장점은 **새로운 로봇에 빠르게 적응**할 수 있다는 것이다. 2.6절에서 설명한 LoRA를 적용하면 전체 7B 파라미터 중 **약 1%만 업데이트**하여 새 로봇의 행동 공간과 시각적 환경에 맞출 수 있다.

$$\text{학습 파라미터} = \frac{r \times (d_{\text{in}} + d_{\text{out}}) \times L}{d_{\text{in}} \times d_{\text{out}} \times L} \approx \frac{2r}{d} \approx 1\%$$

여기서 $r = 16$ (LoRA 랭크), $d = 4096$ (모델 차원), L = 레이어 수.

2.11 pi0 — Flow Matching으로 행동을 생성하는 이유

RT-2와 OpenVLA는 행동을 **이산화(discretize)**하여 분류 문제로 풀었다. pi0는 근본적으로 다른 접근법을 취한다: **연속 행동을 Flow Matching으로 직접 생성**한다.

이산화의 문제점: 연속 행동 $a \in [-1, 1]$을 256개 구간으로 나누면 해상도가 $\frac{2}{256} \approx 0.0078$이다. 대부분의 과제에는 충분하지만, 정밀한 조립이나 삽입(insertion) 과제에서는 이 양자화 오차가 문제가 된다. 또한 이산화는 **다차원 간의 상관관계를 무시**한다 — 각 차원을 독립적으로 분류하기 때문이다.

pi0의 핵심 아이디어: 행동을 노이즈에서 시작해서 **점진적으로 깨끗한 행동으로 변환**하는 생성 모델을 사용한다. 이것은 이미지 생성에서 Stable Diffusion이 노이즈에서 이미지를 만드는 것과 같은 원리다.

왜 Diffusion 대신 Flow Matching인가?

	Diffusion	Flow Matching
수학적 기반	SDE (확률 미분방정식)	ODE (상미분방정식)
노이즈→데이터 경로	구불구불 (곡선)	**직선**
필요 스텝 수	50~1000	**5~10**
결정론적	아니오 (확률적)	**예** (ODE)
로봇 실시간 제어	어려움	**가능**

직선 경로의 수학적 의미:

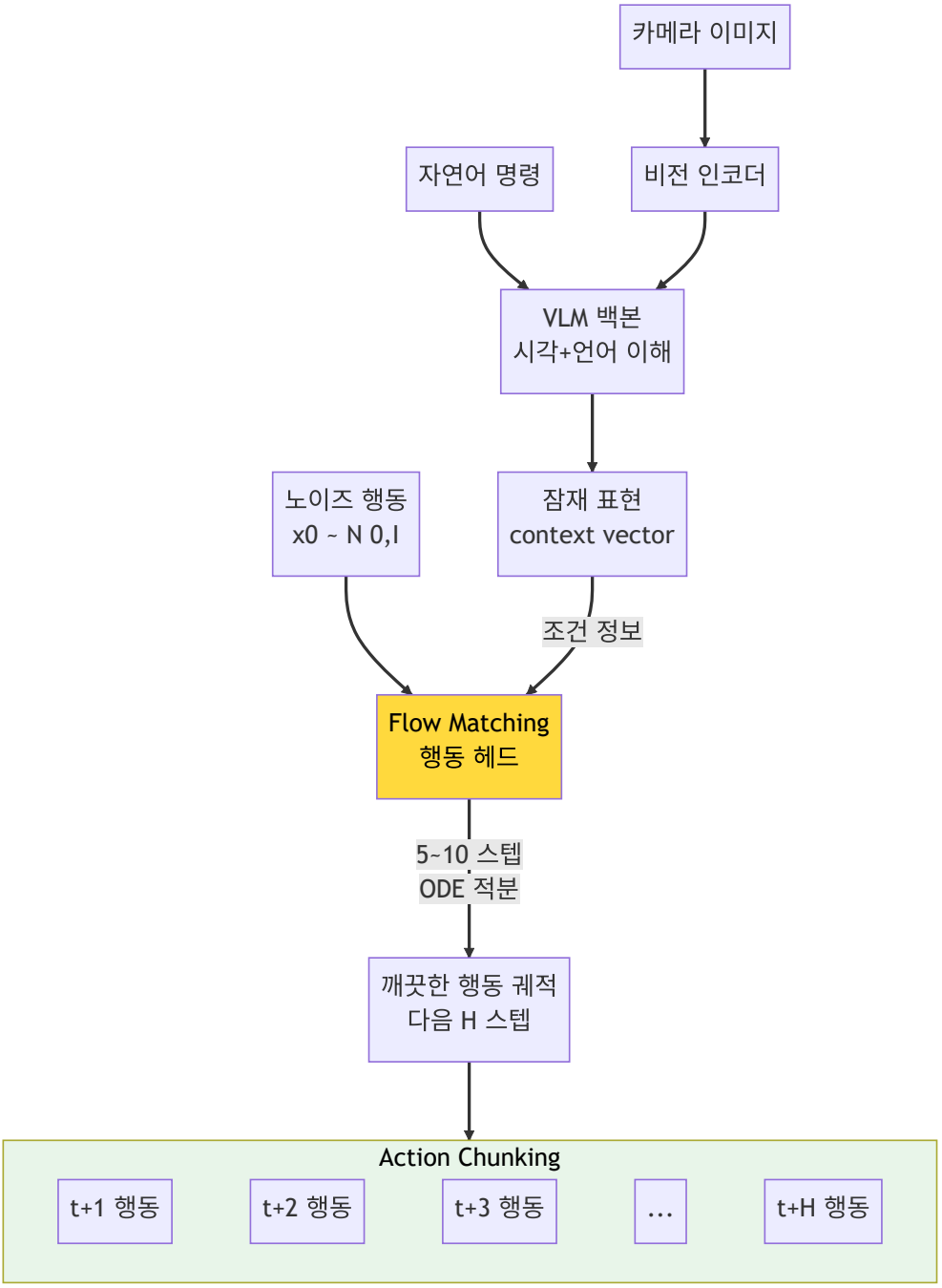
$$\phi_t(x) = (1 - t) \cdot x_0 + t \cdot x_1$$

여기서 x_0은 노이즈, x_1은 목표 행동. 시간 t에서의 위치가 x_0과 x_1의 **선형 보간**이므로, 속도장이 일정하다:

$$v(x, t) = x_1 - x_0$$

이 단순한 속도장이 적은 스텝으로도 정확한 생성을 가능하게 한다.

아키텍처:



Action Chunking — 한 번에 여러 스텝 예측:

기존 모델(RT-1, RT-2)은 **한 번에 1스텝의 행동만 예측**한다. 이것은 두 가지 문제를 야기한다:

1. **누적 오차(compounding error)**: 매 스텝 약간의 오차가 기하급수적으로 누적
2. **비연속적 움직임**: 스텝 간 행동이 부드럽게 연결되지 않음

pi0는 **Action Chunking**을 사용하여 한 번의 추론으로 **다음 H스텝의 전체 행동 궤적**을 생성한다.

$$\text{출력} = [a_{t+1}, a_{t+2}, \ldots, a_{t+H}]$$

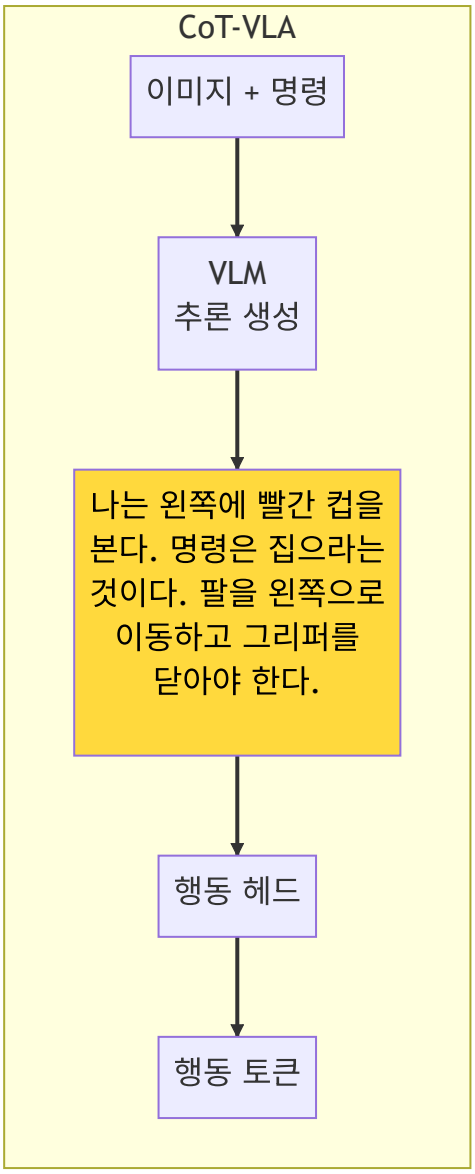
이것은 한 글자씩 말하는 것(매 스텝 예측) 대신 **문장 전체를 한 번에 말하는 것**(궤적 예측)과 같다. 문맥 안에서 각 행동이 서로 일관성 있게 생성된다.

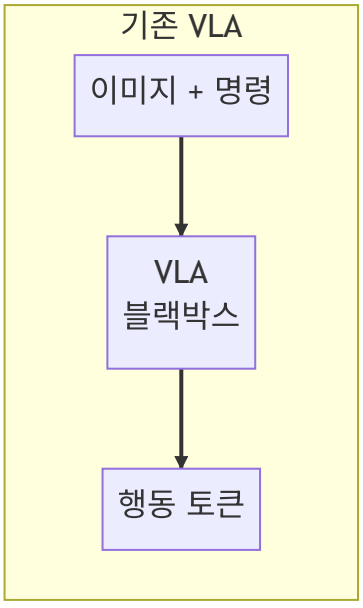
```
# pi0 Flow Matching 행동 생성 (간소화)
def generate_action(model, observation, instruction, num_steps=10):
    # VLM으로 관찰과 명령 인코딩
    context = model.vlm_encode(observation, instruction)
    # 순수 노이즈에서 시작
    action = torch.randn(action_dim)  # 노이즈 행동
    dt = 1.0 / num_steps              # 시간 간격
    for i in range(num_steps):
        t = i / num_steps             # 현재 시간 [0, 1)
        # 속도장 예측: "이 시점에서 어느 방향으로 이동해야 하는가?"
        velocity = model.flow_head(action, t, context)
        action = action + dt * velocity  # 오일러 스텝으로 이동
    return action  # 깨끗한 행동 궤적
```

성능 비교: pi0는 10스텝 Flow Matching으로 약 **50ms** 만에 행동을 생성할 수 있다. 이것은 20Hz 제어 루프에 충분한 속도이며, 50스텝 이상이 필요한 Diffusion 기반 접근법보다 **5~10배 빠르다**.

2.12 CoT-VLA — 로봇도 "생각"할 수 있다

기존 VLA 모델은 **관찰 → 행동**의 직접 매핑이다. 중간 과정이 블랙박스이므로, 왜 그 행동을 선택했는지 알 수 없다. CoT-VLA(Chain-of-Thought VLA)는 이 과정에 **명시적 추론 단계**를 삽입한다.


CoT-VLA
이미지 + 명령
VLM
추론 생성
나는 왼쪽에 빨간 컵을 본다. 명령은 집으라는 것이다. 팔을 왼쪽으로 이동하고 그리퍼를 닫아야 한다.
행동 헤드
행동 토큰


기존 VLA
이미지 + 명령
VLA
블랙박스
행동 토큰

기존 방식 vs CoT-VLA:

	기존 VLA	CoT-VLA
추론 과정	암묵적 (블랙박스)	**명시적** (텍스트 출력)
해석 가능성	불가	**가능** (왜 그 행동인지 읽을 수 있음)
디버깅	어려움	**쉬움** (추론 과정에서 오류 확인)
일반화	패턴 매칭 의존	**논리적 추론** 가능
속도	빠름	**느림** (텍스트 생성 오버헤드)

왜 CoT가 일반화에 도움이 되는가? "빨간 컵을 집어"를 학습한 모델에게 "파란 컵을 집어"를 요청할 때:

- **기존 VLA**: 빨간 컵 패턴과 매칭 실패 → 일반화 불확실
- **CoT-VLA**: "파란 컵이 오른쪽에 있다 → 팔을 오른쪽으로 이동" → 논리적 추론으로 성공

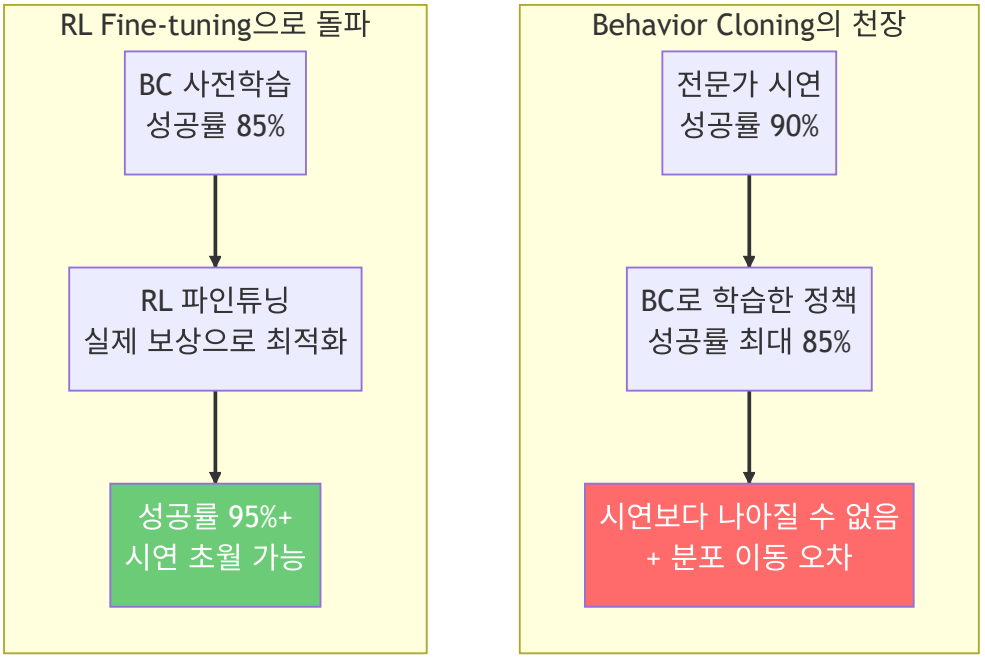
이것은 LLM에서 Chain-of-Thought prompting이 복잡한 문제 해결력을 높이는 것과 **정확히 같은 원리**다. 중간 추론 단계가 입력과 출력 사이의 "간격"을 메워준다.

트레이드오프: CoT-VLA의 대가는 **속도**이다. 행동을 생성하기 전에 텍스트를 먼저 생성해야 하므로, 제어 주파수가 낮아진다. 이것은 정밀한 실시간 조작보다는 **고수준 추론이 필요한 과제**(예: "가장 건강한 음료를 골라 집어")에 더 적합하다.

Part 3 보충: RL과 시뮬레이션 심화

3.8 RL Fine-tuning — 왜 순수 모방학습으로는 부족한가?

2.1절에서 Behavior Cloning의 distributional shift 문제를 설명했다. 여기서는 더 근본적인 한계를 다룬다: **모방학습은 시연자의 수준을 초월할 수 없다.**



AlphaGo의 교훈: AlphaGo는 처음에 인간 기보로 모방학습(SL)을 했다. 하지만 세계 챔피언을 이긴 것은 **이후의 RL 자기 대전**(self-play) 덕분이다. 모방학습은 "합리적인 출발점"을 제공했고, RL이 그 이상으로 끌어올렸다.

로봇에서도 동일한 파이프라인:

$$\text{BC 사전학습 (warm start)} \xrightarrow{\text{합리적 초기 정책}} \text{RL 파인튜닝} \xrightarrow{\text{보상 최적화}} \text{최적 정책}$$

1. **BC 사전학습**: 시연 데이터로 "대략 맞는" 정책을 학습 (무작위보다 훨씬 나음)
2. **RL 파인튜닝**: 실제 과제 보상을 최적화하여 정책을 개선

RLHF와의 유비: ChatGPT의 학습 과정과 정확히 대응된다:

단계	ChatGPT	로봇
1단계	웹 텍스트로 사전학습	시연 데이터로 BC
2단계	SFT (지시 따르기 학습)	과제별 파인튜닝
3단계	**RLHF** (인간 선호 최적화)	**RL** (과제 보상 최적화)

RL Fine-tuning의 주요 난제:

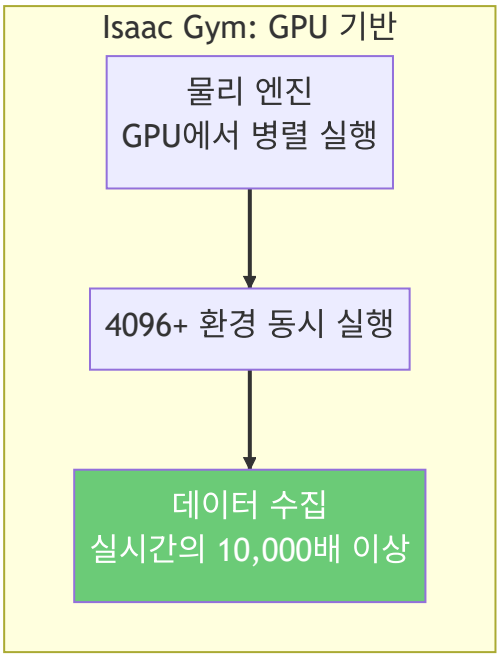
문제	설명	대응
보상 해킹	보상의 허점을 찾아 과제를 풀지 않고 점수만 올림	보상 함수 정교화, 제약 조건 추가
치명적 망각	RL 중 BC에서 배운 기본 능력을 잊음	KL 페널티: $r' = r - \beta D_{KL}(\pi \| \pi_{\text{BC}})$

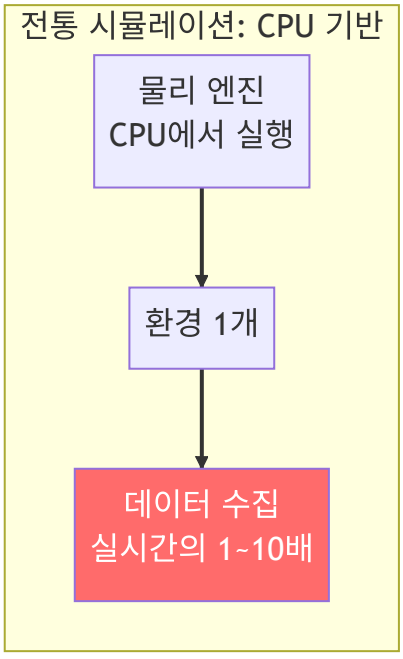
샘플 효율성	RL은 대량의 환경 상호작용이 필요	시뮬레이션 활용 (3.9절)

3.9 Isaac Gym/Sim — 왜 GPU 시뮬레이션이 게임 체인저인가?

RL의 근본적 문제는 **데이터 효율성**이다. 하나의 정책을 학습하는 데 수백만~수억 스텝의 환경 상호작용이 필요하다. 현실 세계에서 이것은 불가능하다.

전통적 시뮬레이션의 한계:





핵심 혁신: Isaac Gym(NVIDIA)은 물리 시뮬레이션 자체를 **GPU에서 실행**한다. 기존에는 CPU에서 물리를 계산하고, GPU에서 정책을 실행하며, 데이터를 CPU↔GPU 사이에서 옮겨야 했다. Isaac Gym은 **물리, 렌더링, 정책 추론, 학습 모두 GPU 메모리 안에서** 처리한다.

속도 비교:

방법	환경 수	속도	1년치 경험 소요 시간
실제 로봇 1대	1	1x (실시간)	1년
CPU 시뮬레이션	1~32	10~100x	며칠~몇 주
Isaac Gym (GPU)	**4096+**	**10,000x+**	**약 1시간**

$$1\ \text{GPU 시간} \approx 1\text{년의 현실 세계 경험}$$

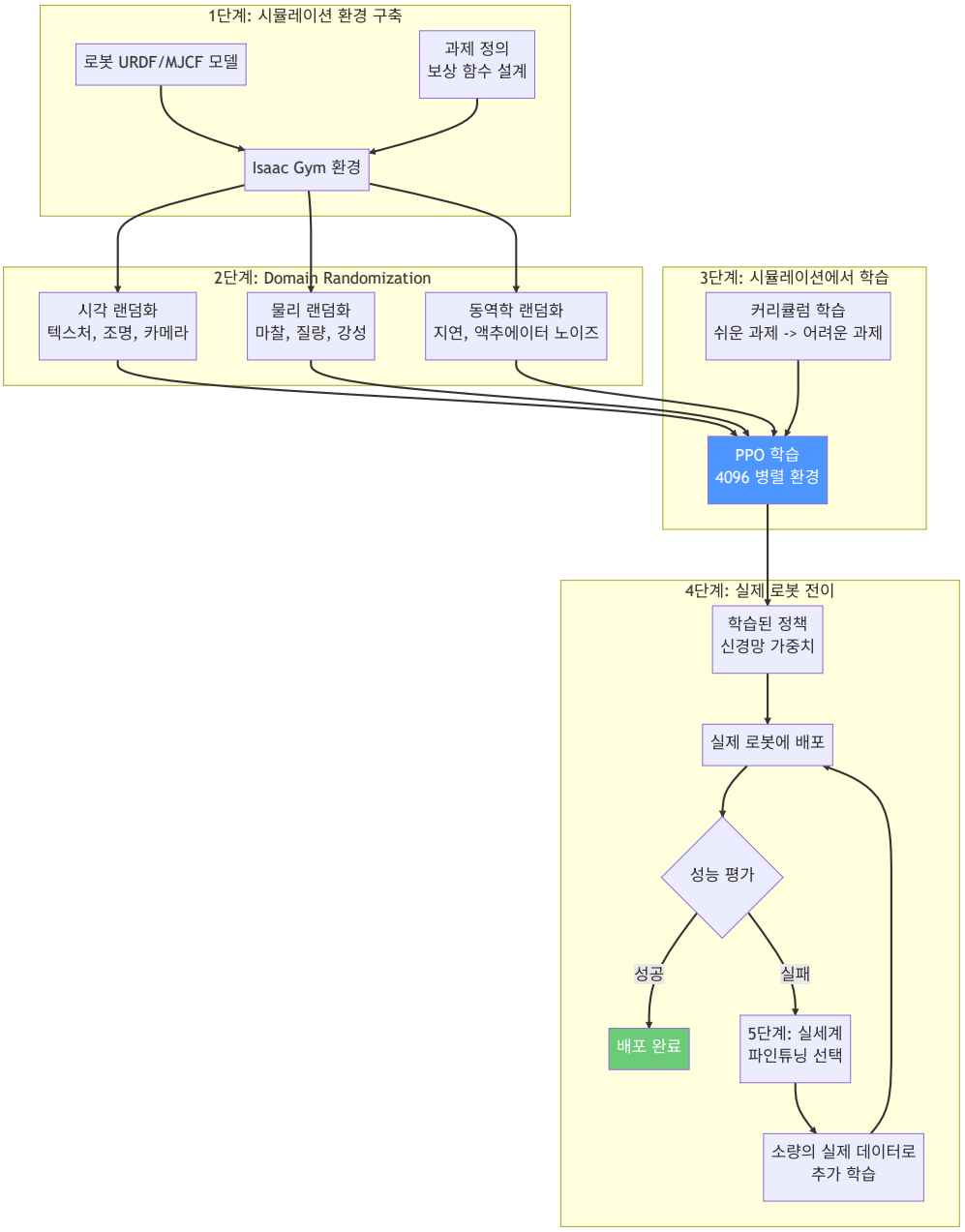
이것이 왜 **게임 체인저**인가: RL은 더 이상 "실제 로봇으로는 불가능한 기법"이 아니다. GPU 시뮬레이션에서 수십 년치 경험을 몇 시간 만에 쌓고, 학습된 정책을 실제 로봇으로 전이(transfer)하면 된다.

Isaac Gym이 가능하게 한 것들:

- **휴머노이드 보행**: 수억 스텝의 RL → 수 시간 만에 학습
- **민첩한 손 조작 (dexterous manipulation)**: 1000개+ 환경에서 동시 학습
- **Domain Randomization**: 4096개 환경에 서로 다른 물리 파라미터 → 자연스러운 랜덤화

3.10 Sim-to-Real의 실제 파이프라인

시뮬레이션에서 학습한 정책을 현실 세계로 전이하는 전체 과정을 체계적으로 정리한다.



Domain Randomization의 세 축:

1. 시각 랜덤화 (Visual Randomization):

- 텍스처: 물체, 바닥, 벽의 색상/패턴을 랜덤 변경
- 조명: 광원 위치, 강도, 색온도 랜덤
- 카메라: 위치, 각도, FOV 미세 변동
- 목적: 정책이 특정 시각적 외형에 과적합하지 않도록 함

2. 물리 랜덤화 (Physics Randomization):

- 마찰 계수: $\mu \in [0.2, 1.0]$ (유리처럼 미끄러운 것 ~ 고무처럼 거친 것)
- 물체 질량: $m \in [0.5m_0, 2.0m_0]$ (기준 질량의 0.5~2배)
- 관절 감쇠(damping): 로봇 관절의 저항 변동

- 목적: 어떤 물리 조건에서든 작동하는 **강건한(robust) 정책**

3. 동역학 랜덤화 (Dynamics Randomization):

- 제어 지연(latency): $\Delta t \in [0, 40\text{ms}]$ (통신 지연 모사)
- 액추에이터 노이즈: 명령한 토크와 실제 출력 사이의 오차
- 관찰 노이즈: 센서 측정값에 가우시안 노이즈 추가
- 목적: 현실의 불완전한 하드웨어를 견디는 정책

커리큘럼 학습 (Curriculum Learning): 처음부터 어려운 과제를 주면 RL이 학습 신호를 찾지 못한다. 점진적으로 난이도를 올린다:

$$\text{난이도}(n) = \min\left(1.0,\ \frac{\text{성공률}(n-1)}{0.8}\right)$$

예: 보행 학습 시 — 평지 → 약간 기울어진 면 → 계단 → 울퉁불퉁한 지형

시스템 식별 (System Identification): Domain Randomization의 범위를 **현실 로봇의 실제 물리 파라미터를 측정하여** 설정한다.

$$\text{랜덤화 범위} = [\hat{\theta} - k\sigma,\ \hat{\theta} + k\sigma]$$

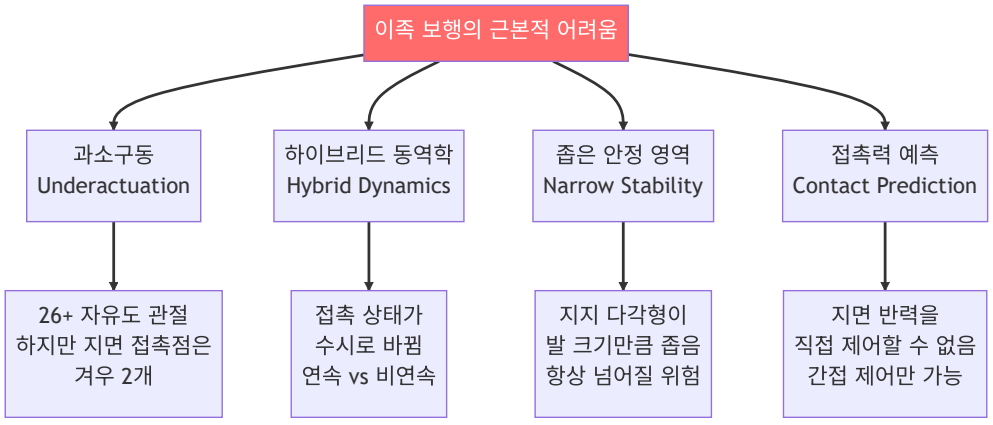
여기서 $\hat{\theta}$는 측정된 파라미터, σ는 측정 불확실성, k는 안전 마진(보통 2~3).

핵심 가정: **현실의 물리 파라미터가 랜덤화 범위 안에 포함**되어야 한다. 이것이 보장되면 시뮬레이션에서 학습한 정책이 현실에서도 작동할 가능성이 높다.

3.11 휴머노이드 보행 제어 — 왜 가장 어려운 문제인가?

이족 보행은 로봇 공학에서 **가장 도전적인 문제** 중 하나이다. 3.6절에서 ZMP와 캡처 포인트를 간략히 다뤘지만, 여기서는 왜 이 문제가 본질적으로 어려운지를 더 깊이 살펴본다.

근본적 어려움:



과소구동 (Underactuation): 휴머노이드는 약 26개 이상의 능동 관절(자유도)을 가지지만, 지면과의 접촉점은 **겨우 2개(양 발)**이다. 로봇의 무게 중심(CoM)을 직접 제어할 수 없고, 관절 토크를 통해 **간접적으로만** 제어할 수 있다.

$$\underbrace{M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q)}_{\text{로봇 동역학}} = \underbrace{\tau}_{\text{관절 토크}} + \underbrace{J_c^T f_c}_{\text{접촉력}}$$

여기서 접촉력 f_c는 제어할 수 없는 외부 힘이다.

하이브리드 동역학 (Hybrid Dynamics): 보행 중 동역학 모델이 **불연속적으로 전환**된다:

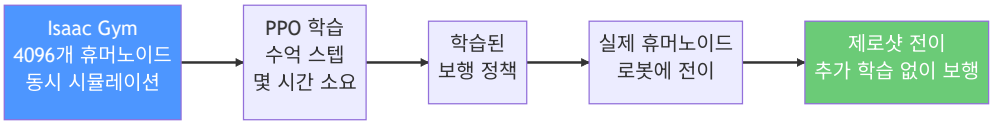
- 두 발 접촉 (double support) → 한 발 접촉 (single support) → 두 발 모두 공중 (flight, 달릴 때)
- 각 단계에서 동역학 방정식이 다르다
- 전환 순간에 **충격(impact)**이 발생한다

사족 보행 vs 이족 보행 비교:

특성	사족 보행 (개 로봇)	이족 보행 (휴머노이드)
접촉점	4개	2개
정적 안정성	넓은 지지 다각형	**좁은 지지 다각형**
한 발 들어도 안정?	3점 지지 → 안정	**불안정** (연속 제어 필요)
넘어지면?	쉽게 복구	**치명적** (높은 무게 중심)
자유도	~12	**~26+**
에너지 효율	높음	낮음 (불안정 보상에 에너지 소모)
RL 학습 난이도	중간	**매우 높음**
Sim-to-Real 성공	2019년부터	2023년부터

현대적 접근: Sim-to-Real RL:

Tesla Optimus, Figure 01/02, Unitree H1/G1 등 최신 휴머노이드 로봇은 모두 **시뮬레이션에서 RL로 보행을 학습**하는 접근법을 사용한다.



보행 보상 함수 설계: RL 학습의 핵심은 보상 함수이다. 휴머노이드 보행의 보상은 여러 항의 **가중합**으로 구성된다:

$$r = w_1 \cdot r_{\text{speed}} + w_2 \cdot r_{\text{energy}} + w_3 \cdot r_{\text{alive}} + w_4 \cdot r_{\text{smooth}} + w_5 \cdot r_{\text{limits}}$$

보상 항	수식	의미
r_{speed}	$\exp(-\lvert v - v_{\text{target}}\rvert^2)$	목표 속도 추종
r_{energy}	$-\sum \lvert\tau_i \cdot \dot{q}_i\rvert$	에너지 효율 (작을수록 좋음)
r_{alive}	$+1$ (안 넘어지면)	생존 보너스
r_{smooth}	$-\lvert\ddot{q}\rvert^2$	부드러운 움직임 (관절 가속도 최소화)
r_{limits}	$-\sum \max(0, \lvert q_i\rvert - q_{\max})$	관절 한계 이내 유지

제로샷 전이(Zero-Shot Transfer)의 성공: 최근 연구에서 놀라운 결과는, 시뮬레이션에서만 학습하고 **실제 로봇에서 추가 학습 없이** 바로 보행에 성공한다는 것이다. 이것이 가능한 이유:

1. **충분한 Domain Randomization**: 마찰, 질량, 지연 등을 넓게 랜덤화
2. **대규모 병렬 학습**: 4096개 환경에서 다양한 조건을 동시에 경험
3. **관찰 공간 설계**: 카메라 이미지 대신 **관성 센서(IMU) + 관절 각도**를 입력으로 사용 → 시각적 domain gap 회피
4. **지형 커리큘럼**: 평지 → 경사 → 계단 → 불규칙 지형으로 점진적 학습

이것은 로봇 공학의 패러다임 전환이다. 과거에는 수학적 모델(ZMP, 역동역학)을 기반으로 수년간 제어기를 설계했지만, 이제는 **보상 함수만 잘 설계하면 RL이 알아서 보행을 학습**한다.